



Servlet IV

1. Handling sessions
2. Introduction to servlet filters
3. Creating simple filter

- HttpSession API is a high-level interface that allows the server to "remember" a set of information relevant to a particular user's on-going transaction.
- Web server will assign a unique "session id" to the session(memory block) and send this id to the browser, asking the browser to send back the id in any future requests.
- By default cookies will be used to send this session id to the client. If cookies are disabled, URL can be used by the server to send the session id.
- Session Creation
 - Request object provides the method getSession() for getting the session. Web server will either create the session newly or return the existing one

- Storing Data in Session
 - `setAttribute(name, value)` method of session object can be used to store the necessary information.
- Retrieving Data from Session
 - `getAttribute(name)` method of session object can be used to retrieve the information. This method returns Object which needs to be downcasted.
- Session Timeout
 - Sessions expire after a certain period of inactivity.
 - Session timeout can be set for an individual session or in `web.xml`
- Destroying a Session
 - Session can be manually destroyed using `session.invalidate()`

- Filters are a way to implement common tasks that apply to multiple servlets.
- Filters are pluggable classes that stand between the client and a target component.
- Multiple filters can be configured in a chain each performing specific task before it moves to next.
- Filter lifecycle
 - It is similar to servlet life cycle.
 - Service method in servlet life cycle is similar to `doFilter()` method in filter life cycle which gets executed for every request.

- A Filter class can be created by implementing Filter interface.
- `init()` and `destroy()` method can be used for initialization and cleanup code
- `doFilter()` method can be implemented like service method in servlet
- This filter class can be annotated with `@WebFilter` annotation to specify which URL pattern it gets associated with.

Example:

If filter is annotated with `@WebFilter("/*")`, it gets executed for all the requests received from the client.